

Protocol-Governed Systems: Compiler Conceptual Model

(c) 2026 Bhash Ganti

Contact: <mailto:bachipeachy@gmail.com>

ORCID Profile: <https://orcid.org/0009-0007-3810-6520>

Preface

This paper is part of PGS technical paper series. The paper, *Protocol-Governed Systems: Conceptual Model*, established the architectural foundations: constitutional governance, the separation of governance from execution, and the four-layer stack that makes governed execution possible. This paper focuses on one component of that stack: the PGS compiler.

The compiler is the mechanism that converts governance declarations into a structure that execution can consume. Understanding what the compiler does, what it produces, and what it guarantees is essential to understanding how PGS works. No prior knowledge of compilers is assumed. The paper is written for readers who understand the PGS conceptual model and want to understand how its central promise — governance before execution — is actually delivered.

Abstract

Protocol-Governed Systems (PGS) separate governance from execution. Governance determines what behavior is allowed to exist. Execution performs only behavior that governance has already admitted. The compiler is the component that makes this separation possible.

Traditional compilers translate source code into executable form. The PGS compiler serves a different purpose. It transforms protocol declarations into a governed execution boundary called the Protocol Snapshot. The snapshot is not a program. Instead, it is a complete description of every execution path that governance has admitted. The runtime consumes this snapshot and executes only within its boundaries.

This paper defines the conceptual model of the PGS compiler. It explains why the compiler exists, what problem it solves, how it constructs admissibility before execution begins, why the compiler's internal model is a semantic graph rather than a syntax tree, and how a single semantic model produces multiple snapshot projections for different consumers. The paper also defines the compiler's boundary contract: what guarantees a compiled snapshot provides, what guarantees it does not provide, and why the compiler is the primary enforcement boundary in Protocol-Governed Systems.

1. Introduction

The central idea behind Protocol-Governed Systems is simple: governance must be resolved before execution begins. Most systems operate in the opposite order. Behavior executes first, and governance mechanisms attempt to observe, constrain, audit, or correct that behavior after it has already occurred. As systems become larger, more autonomous, and increasingly AI-generated, this model becomes difficult to sustain. Governance mechanisms struggle to keep pace with implementation velocity.

PGS addresses this problem by moving governance earlier. Instead of asking *should this behavior be allowed?* at runtime, PGS asks *should this behavior be allowed to exist?* at compile time. This seemingly small shift has large consequences. If an execution path is not constitutionally admissible, it is not blocked by runtime guards — it is **never constructed**. The runtime **never sees it**. The runtime **cannot execute it**.

This creates a new architectural requirement. Governance declarations alone are insufficient. A system needs a mechanism that transforms governance declarations into a structure that execution can consume. The PGS compiler is that mechanism. Its responsibility is not translation — it is construction. The compiler takes protocol declarations and produces a Protocol Snapshot: a complete description of the admissible execution surface of the system. Everything inside the snapshot may execute. Everything outside the snapshot is structurally absent. The compiler is therefore the component that converts governance from documentation into executable structure. Without the compiler, governance remains declarative. With the compiler, governance becomes the execution boundary.

2. What the Compiler Produces

The easiest way to understand the compiler is to begin with its output. Traditional compilers produce executable artifacts: source code enters, an executable program emerges. The PGS compiler produces a different result. Protocol declarations enter, and a Protocol Snapshot emerges. The snapshot is not executable code. It is a governed execution map.

The snapshot contains all admissible workflows, all capability bindings, all dependency relationships, all machine addresses, all governance projections, and all compiler evidence. The snapshot is the only artifact consumed by the runtime. The runtime never reads protocol declarations directly. The runtime consumes the snapshot and executes only within its declared topology. This creates a strict separation of responsibilities. The compiler determines what may exist. The runtime determines what happens during execution.

The compiler governs possibility. The runtime governs realization.

This separation is one of the most important architectural properties of PGS.

Figure 1 shows the compiler’s functional model: what it takes as input, the internal process it performs, and the multiple projections it produces as output.

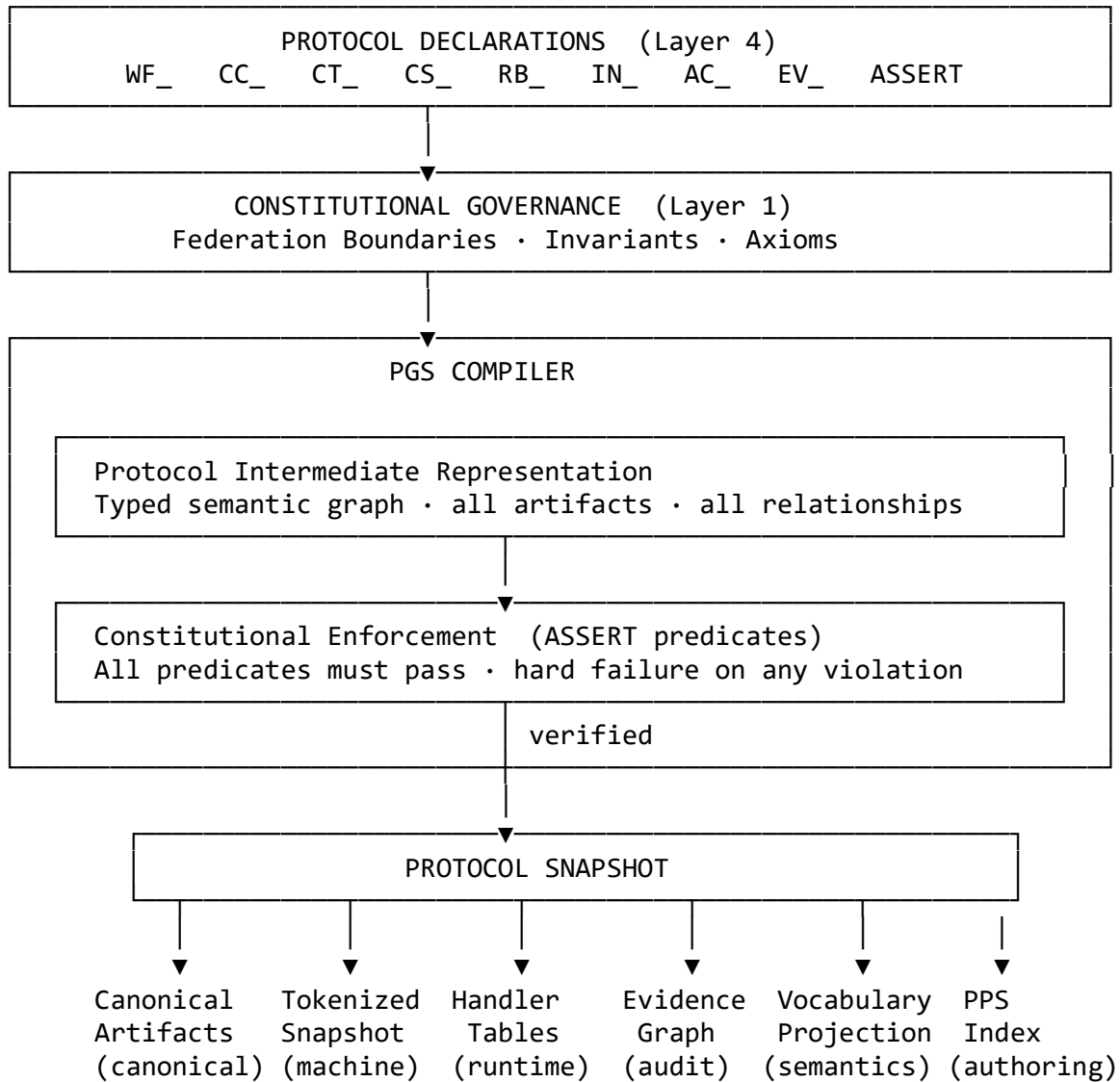


Figure 1: PGS Compiler Functional Model. Protocol declarations and constitutional governance enter the compiler. The compiler builds a typed semantic graph, enforces all constitutional predicates, and — only if every predicate passes — produces the Protocol Snapshot as six orthogonal projections from the same verified model.

3. Why the Runtime Is Simple

Understanding what the compiler produces explains something that initially surprises many readers: the PGS runtime is intentionally simple. A traditional runtime carries significant responsibility. It enforces access controls, validates inputs against business rules, checks authorization before sensitive operations, and audits behavior after the fact. This is necessary because behavior was not governed before execution began — the runtime must compensate.

In PGS, the compiler has already resolved all of this. By the time execution begins, the runtime has a snapshot that answers every governance question in advance. Which workflows are admissible? Which capability paths are permitted? Which operations are constitutionally allowed? All answers are already present in the snapshot. The runtime does not need to ask these questions. It reads the topology, traverses the declared graph, and executes only what the snapshot permits. It has no domain knowledge. It enforces no governance logic. It simply executes a structure that governance has already admitted.

This is not a limitation — it is a design property. Runtime simplicity is the direct consequence of compile-time completeness. The compiler absorbs governance complexity so the runtime does not have to. The runtime is intentionally incapable of governance.

4. The Semantic Model

To construct a snapshot, the compiler needs more than a list of artifacts. It needs to understand relationships. It needs to reason about dependency: which capability contract references which transform. It needs to reason about authority: which workflow governs which execution path. It needs to reason about admissibility: which combinations of artifacts satisfy constitutional requirements. A syntax tree is not sufficient for this. A syntax tree answers *what does this declaration say?* The compiler needs to answer *what does this declaration mean in relation to everything else?* For this, the compiler builds a semantic graph.

4.1 The Protocol Intermediate Representation

The Protocol Intermediate Representation is the compiler's internal model of the entire protocol. It is not stored anywhere in the snapshot — it is the structure the compiler uses while building the snapshot. Every protocol artifact becomes a node in the semantic graph. Every relationship between artifacts becomes a typed edge. The semantic graph is the compiler's source of truth during compilation. Every projection produced by the compiler is derived from the semantic graph. The relationships in the semantic graph are captured as typed edges, and these typed edges are not generic. `CC_BINDS_CT` means this capability contract binds to this capability transform. `WF_CONTAINS_NODE` means this workflow contains this capability contract as an execution step. `CC_GOVERNED_BY` means this capability contract is governed by this assertion. Crucially, these typed edges are not inferred or discovered by the compiler; rather, they are explicitly declared by the protocol, and the compiler's role is to resolve and validate them.

This distinction is important. In a traditional compiler, the compiler infers relationships from syntax. In PGS, every relationship is explicitly declared, and the compiler validates that the declared relationships are real, complete, and constitutionally admissible. If a workflow references a capability contract that does not exist, the compiler fails. There is no fallback and no partial snapshot. If a capability contract references an operation that is not in the constitutional vocabulary, the compiler fails. Everything must be present, declared, and admissible before the snapshot is produced.

4.2 Why a Semantic Graph

The semantic graph has properties that matter for governance. First, it is traversable from any direction. The compiler can ask which workflows depend on a given capability contract, and it can also ask which assertions govern that same contract — both questions answered by traversing edges. Second, it is complete before any projection is produced. The compiler does not project artifacts one at a time. It first builds the entire semantic model, verifies it against constitutional requirements, and then produces multiple projections from the verified model. Third, it supports evidence generation. Because every relationship is typed and resolved, the compiler can record exactly why each artifact was admitted. This record becomes the evidence graph, described in Section 6.

5. Governance as Executable Law

The PGS compiler enforces governance, but governance in PGS is not a set of style guidelines or a checklist reviewed by humans before deployment. It is executable constitutional law.

5.1 The ASSERT Model

Constitutional requirements are declared as ASSERT artifacts. Each ASSERT artifact defines a predicate that the semantic graph must satisfy. If the predicate is not satisfied, compilation fails and the snapshot is not produced. For example: every capability contract must declare only storage operations from the enumerated constitutional vocabulary; every runtime binding must reference a capability contract that exists in the snapshot; every workflow step must have a declared outcome that routes the execution graph. These predicates are not warnings and are not configurable thresholds. They are **hard failures**.

The constitutional vocabulary of storage operations is declared once. Every capability contract in every domain is checked against it. If any contract references an operation outside the vocabulary, no snapshot is produced. This is the mechanism that converts governance from intention into enforcement. A governance requirement expressed as an ASSERT artifact is not documentation — it is a structural gate that every protocol declaration must pass before execution is permitted to exist.

5.2 Constitutional Enforcement at Scale

One ASSERT artifact governs across the entire protocol. It does not need to be repeated per domain or manually applied per artifact. It traverses the semantic graph and checks every node of the relevant type. This is why constitutional coverage does not grow with the size of the system. A new domain is automatically subject to every existing ASSERT artifact. A new protocol version is automatically checked against every existing constitutional requirement. The compiler is the mechanism that makes this scaling possible.

6. Compile-Time Evidence

When the compiler admits a protocol artifact, it records why. This record is not a log and not a side effect — it is a first-class compiler output called the evidence graph.

6.1 What the Evidence Graph Contains

The evidence graph is a projection of the semantic graph that captures admissibility reasoning. For each admitted artifact, it records which artifacts it depends on, which assertions it satisfied, which constitutional requirements it cleared, and which artifacts it governs or is governed by. This is the audit trail of compile-time governance. It is produced at compile time and readable at any time after.

6.2 Why Compile-Time Evidence Matters

Traditional systems produce audit trails at runtime: an action occurs, a log entry is written, and the log describes what happened. This is retrospective evidence. PGS produces evidence before execution begins. The evidence graph documents not what happened, but what was admitted and why. This is prospective evidence. The difference is structural:

Traditional audit: Execution → Evidence

PGS evidence: Admissibility → Evidence → Execution

It answers a different question — not *did this execution conform to governance?* but *was this execution topology constitutionally admissible before it was ever run?*

For autonomous systems and AI-generated behavior, this distinction is significant. Retrospective audit trails can document violations after they occur. Prospective evidence eliminates entire classes of violation before execution begins.

7. One Semantic Model, Multiple Projections

The semantic graph is built once. The snapshot is produced from it in multiple forms for multiple consumers. This is one of the least obvious but most consequential properties of the PGS compiler.

7.1 The Problem of Multiple Consumers

Different consumers of the snapshot need different views. The runtime needs a machine-efficient representation of execution topology. The governance layer needs handler tables mapping protocol declarations to concrete implementations. A developer inspecting the system needs human-readable artifact descriptions. A monitoring tool needs a vocabulary index of every named concept in the system. A dependency tracker needs the evidence graph. If each of these views were built from different sources, they would diverge. A change in one place would require coordinated updates everywhere, and inconsistency would be the default rather than the exception.

7.2 Single Source, Multiple Projections

The PGS compiler solves this by producing all views from a single semantic model. Every projection — canonical artifacts, tokenized runtime topology, handler and dispatch tables, the evidence graph, the vocabulary projection, and the protocol projection surface — is derived from the same verified semantic graph. They cannot diverge. A change to a protocol declaration rebuilds the semantic graph, and all projections are regenerated from the updated model. There is no partial update and no coordinated synchronization. There is one source of truth and multiple views of it. This is what makes the snapshot a governed execution map rather than a collection of files.

7.3 Canonical Artifacts

The canonical artifact projection is the human-readable, authoritative form of each protocol declaration. It contains every workflow, capability contract, capability transform, capability side effect, runtime binding, intent, actor, and event in the protocol, expressed as structured documents indexed by fully qualified domain name. This projection is the authoritative record of what the protocol says. It is the form that developers read, governance teams review, and auditors inspect. It is produced directly from the verified semantic graph and never modified by hand. Any edit to a canonical artifact requires re-authoring the protocol source and recompiling — the projection is never the source of truth.

7.4 Tokenized Snapshot

The tokenized snapshot is a machine-optimized projection of the same protocol topology. Tokenization exists to decouple governance from execution substrate. Every string identifier in the canonical representation — workflow names, capability contract codes, outcome labels, field names — is replaced by a compact numeric token derived from its

fully qualified domain name at compile time. The topology itself is identical. The representation is stripped to its minimum footprint.

The significance of this projection is not merely performance. It opens an architectural possibility that canonical JSON cannot: the tokenized snapshot can be consumed by execution substrates that have no string processing capability at all. A conventional software runtime reads the canonical snapshot. A constrained embedded controller can read the tokenized snapshot. A hardware accelerator — an FPGA or application-specific integrated circuit — can evaluate the execution topology directly from the token table, with governance-defined routing encoded as numeric comparisons rather than string lookups.

This matters architecturally because it decouples governance from substrate. The constitutional model, the protocol declarations, and the compilation process remain unchanged. Only the projection consumed by the execution substrate changes. A system governed under the same ASSERT predicates and the same federation boundaries can execute identically on a general-purpose server, on a constrained IoT device, or on silicon — because the execution topology is fully resolved at compile time and the tokenized projection carries no interpretation overhead. The runtime in each case is a reader of a static, verified structure. The structure is the same. The substrate consuming it can vary without any change to governance.

7.5 Handler and Dispatch Tables

The handler table projection maps each declared capability contract to the concrete implementation that satisfies it at runtime. The dispatch table maps each workflow step to its resolved execution address. These projections are the binding layer between the governed protocol topology and the implementation code that realizes each capability. The runtime reads the handler table to locate implementations; it never resolves capability identities dynamically. This ensures that implementation lookup is a compile-time-verified operation, not a runtime search. If an implementation is missing when the snapshot is built, the compiler fails — the handler table cannot be produced with gaps.

7.6 Vocabulary Projection

The vocabulary projection is an aggregate semantic index of every named concept in the entire protocol. Across all domains and all subdomains, every artifact code, every outcome label, every storage operation, and every field name is indexed by its fully qualified domain name. This projection serves multiple consumers. A governance author uses it to verify that a new term does not duplicate or shadow an existing one. A static analysis tool uses it to trace semantic references across domain boundaries. A documentation system uses it to generate a complete glossary of the system's governed vocabulary. The vocabulary projection is the compile-time proof that the protocol's address space is coherent and non-ambiguous.

7.7 Protocol Projection Surface

The Protocol Projection Surface (PPS) is an authoring-oriented index of the compiled protocol. It presents the governance structure of the system — its subdomains, its capability surfaces, its declared behaviors — organized as a navigation surface for the entity constructing new protocol artifacts. Where canonical artifacts answer the question *what does this artifact say?*, the PPS answers the question *what exists in this protocol, and where does a new artifact belong?*

The PPS is produced from the same semantic graph as every other projection, which means it reflects the current compiled state of the protocol exactly. An author consulting the PPS is consulting the compiler's own verified model of the governance surface. This makes the PPS the correct input to governed authoring workflows: the human or agent constructing a new protocol artifact begins from a view that the compiler itself has certified as accurate.

This property enables a structured authoring strategy built around two explicitly defined concepts. The first is *Business Intent*: a human-authored statement of what a new subdomain or capability should do, expressed in governance terms without reference to implementation. The second is *Governance Intent*: the machine-assisted translation of that business goal into governed protocol structure — the intermediate artifact between human goals and compiled canonical artifacts. An author working from the PPS can move from Business Intent to Governance Intent without needing to understand the full protocol implementation, because the PPS provides a complete, compiler-verified surface of what already exists. The authoring steps require only an understanding of what the new artifact should do and where in the governance surface it belongs. The protocol's full implementation complexity is already resolved. The author works only at the governance boundary. This authoring pipeline is the subject of Paper 6 in this series.

7.8 Execution Graph Before Execution

Because all execution topology is determined at compile time, a complete, traversable execution graph of every admitted workflow exists before any execution ever starts. This is not a simulation and not an approximation. It is the literal structure the runtime will traverse — made available for inspection, analysis, and visualization prior to the first request.

This property has consequences that go beyond debugging. A governance reviewer can inspect the complete reachability graph of a workflow — every possible path, every possible outcome, every terminal state — before the system is deployed. A test architect can derive a complete test matrix from the execution graph without executing any implementation. A security analyst can verify that no path leads to an unauthorized capability without running the system. A compliance officer can confirm that every execution path satisfies a given requirement by traversing the graph rather than observing runtime logs.

In conventional systems, this kind of analysis requires either static analysis tools applied to implementation code — with the attendant uncertainty about dynamic behavior — or exhaustive runtime testing. In PGS, the execution graph is a first-class compiler output. It reflects not what the implementation might do, but what the protocol has declared is admissible. The graph is the governance artifact. Analysis of the graph is analysis of the governance.

8. The Admissibility Boundary Contract

The compiler makes guarantees. Understanding what those guarantees are — and what they are not — is essential to understanding how PGS allocates responsibility across its components.

8.1 What the Compiler Guarantees

When the compiler produces a snapshot, it provides four guarantees. **Constitutional admissibility**: every artifact in the snapshot has satisfied every ASSERT predicate declared in the constitutional governance layer; no artifact violated a constitutional requirement. **Structural completeness**: every reference in the snapshot resolves; no workflow references a missing capability contract, no capability contract references a missing transform, and no runtime binding references an artifact that does not exist. **Semantic addressing**: every artifact has a stable, unique machine address derived from its fully qualified domain name; addresses are deterministic, and identical inputs produce identical addresses. **Topological integrity**: every execution path has declared outcomes, every outcome routes to a declared next step or terminal state, and there are no dead ends or undefined outcomes.

These four guarantees are delivered together or not at all. There is no partial snapshot. Either every artifact passes every gate, or no snapshot is produced.

8.2 What the Compiler Does Not Guarantee

The compiler does not guarantee implementation correctness. A capability transform may be structurally admissible and behaviorally incorrect — the compiler validates the declaration, not the implementation. The compiler does not guarantee runtime behavior. A workflow may be constitutionally admissible and still fail at runtime due to data errors, infrastructure faults, or environmental conditions. The compiler governs the structure of execution, not the outcomes of execution. The compiler does not guarantee data validity. Payload values are runtime inputs, and schema conformance for runtime data is a runtime concern.

8.3 Why This Boundary Matters

The boundary between what the compiler guarantees and what it does not is not a limitation — it is a designed separation of concerns. The compiler owns the governance question: is this execution topology constitutionally admissible? The runtime owns the execution question: what happens when this topology is traversed with these inputs? Neither component reaches into the other's domain. This is the same principle — separation of governance from execution — expressed at the component level.

8.4 The Governance Dividend

There is an observable consequence of this separation. As the protocol matures, system evolution becomes more localized. When a governance requirement changes, the

compiler re-evaluates the entire protocol graph, but the runtime does not change. When a new domain is added, the compiler admits it against all existing constitutional requirements, but the runtime does not change.

This creates an observable and non-theoretical asymmetry: governance complexity grows with the protocol, but runtime complexity does not. In the OmniBachi reference implementation, a system-wide change to the identity model — affecting all domains simultaneously — required five localized implementation changes, one structural declaration, and zero modifications to the execution engine. The compiler absorbed the structural consequence of the governance change and projected it into a new snapshot. The runtime consumed the new snapshot without modification. Governance complexity is paid once, at compile time, for all executions. Runtime simplicity is the compound return on that investment.

9. Protocol Inspection

Compilation produces a snapshot, but the snapshot is not the only output of the compiler. The compiler also supports inspection: the ability to query what the protocol means, what it admits, and how its governance surfaces are structured.

Protocol inspection is not a debugging tool — it is an analytical capability. The distinction from compilation is precise: compilation answers *is this protocol admissible?* Inspection answers *what does this protocol mean?* Because inspection traverses the same verified semantic graph that the compiler uses to build the snapshot, its answers carry the same authority as the snapshot itself. An inspection result is not a heuristic or an approximation. It is the compiler’s own semantic model answering a query.

9.1 Governance Surface Queries

Inspection can answer governance surface questions across the entire protocol without recompiling. Which subdomains are present in this snapshot? Which governance requirements apply to this capability contract? Which assertions govern this workflow? Does the current snapshot satisfy the governance intent declared for this subdomain? These queries traverse the semantic graph’s typed edges and return authoritative answers. A developer asking whether a capability contract satisfies a given constitutional requirement does not need to read code — they query the compiler’s model.

9.2 Dependency and Impact Analysis

Because the semantic graph captures all typed relationships, inspection supports precise dependency analysis. Which workflows depend on a given capability transform? What is the full set of artifacts affected if a given storage operation is removed from the constitutional vocabulary? Which capability contracts would fail if a runtime binding were removed? These questions are answerable by graph traversal before any change is compiled. The impact of a proposed governance change can be understood before the change is made.

9.3 Governance Intent Verification

Inspection also supports the governed authoring pipeline. When a governance author has declared the intent for a subdomain — the behavioral goals and constraints for that domain — the compiler can verify whether the compiled protocol satisfies that intent. This is not a natural-language comparison. It is a structural check: does the compiled topology of the subdomain match the declared governance intent for that subdomain? The result is a formal verification that what was authored is what was compiled, surfaced for human review before any artifact is deployed.

This makes inspection the human validation gate in the authoring pipeline. Between the declaration of governance intent and the deployment of compiled artifacts, the compiler’s inspection capability provides the checkpoint at which semantic misalignment can be

caught. The compiler validates syntax and structure. Inspection validates intent alignment. Together, they close the authoring loop.

9.4 The Compiler as Analytical Authority

Taken together, these capabilities establish the compiler as more than a build tool. The compiler is the analytical authority for the protocol. It is the component that can answer questions about the protocol's meaning, its governance surface, its dependency structure, and its intent alignment — with the same authority as the snapshot it produces. Every other tool that analyzes or documents the protocol is secondary to the compiler's model. The compiler's model is the protocol.

10. Conclusion

The PGS compiler is not a traditional compiler. It does not translate source code into executable form. It constructs admissibility. It takes protocol declarations, builds a complete semantic model of the system, verifies that model against constitutional requirements, and produces a Protocol Snapshot: a complete, verified map of every execution path that governance has admitted. The runtime consumes this snapshot, executes only within the declared topology, and enforces no governance logic of its own.

The snapshot is not a single artifact — it is a set of orthogonal projections from one verified semantic model. Each projection serves a different consumer without the possibility of divergence. The canonical artifacts serve human review. The tokenized snapshot serves constrained and hardware execution substrates. The handler tables serve the runtime. The evidence graph serves governance audit. The vocabulary projection serves semantic coherence. The protocol projection surface serves governed authoring. The execution graph, available before the first execution, serves analysis, verification, and test generation. These are not convenience features. They are structural consequences of building from a single verified model.

This is the mechanism behind PGS's central claim. Governance before execution is not a policy — it is an architectural property, enforced by the compiler, delivered as a snapshot, consumed by the runtime. The consequence is a system in which governance scales without runtime complexity growing. New domains are admitted against existing constitutional requirements automatically. New governance requirements apply to existing protocols automatically. The runtime does not change. Governance complexity is paid at compile time. Runtime simplicity is preserved. That is the governance dividend.

Appendix A: Key Terms

Protocol Snapshot: The compiler’s primary output. A complete, verified description of the admissible execution surface of the system. Read-only input to the runtime. Never modified by hand.

Protocol Intermediate Representation: The compiler’s internal semantic model — a typed directed graph where nodes are protocol artifacts and edges are typed relationships. Built during compilation; not persisted in the snapshot. Referred to throughout as “the semantic graph.”

ASSERT Artifact: A constitutional declaration that defines a predicate the semantic graph must satisfy. Evaluated at compile time against all relevant artifacts. Hard failure on violation.

Admissibility: The property of being constitutionally and structurally permitted to exist in the snapshot. The compiler constructs admissibility; it does not assume it.

Projection: A specific view of the snapshot produced from the semantic graph for a particular consumer. Multiple projections are derived from the same verified model and cannot diverge.

Evidence Graph: A compiler-produced projection recording why each artifact was admitted: its dependencies, assertions satisfied, and constitutional requirements cleared.

Tokenized Snapshot: A machine-optimized projection of the protocol topology in which all string identifiers are replaced by compact numeric tokens derived from their fully qualified domain names at compile time. Enables execution on constrained, embedded, and silicon substrates.

Protocol Projection Surface (PPS): An authoring-oriented index of the compiled protocol, presenting the governance structure as a navigation surface for constructing new protocol artifacts.

Vocabulary Projection: An aggregate semantic index of every named concept in the protocol, organized by fully qualified domain name across all domains and subdomains.

Execution Graph: A traversable, compile-time representation of the complete admissible execution topology for each workflow. Available before any execution begins.

Fully Qualified Domain Name (FQDN): The canonical identity of a protocol artifact. Format: `domain::ARTIFACT_CODE_Vn`. Used as the stable addressing key throughout the snapshot and its projections.

Governance Dividend: The observable property that system evolution becomes more localized as governance surfaces mature. The compiler absorbs structural consequence; the runtime remains unchanged.

Business Intent: A human-authored statement of what a new subdomain or capability should do, expressed at the governance boundary without reference to implementation. The starting point of the governed authoring pipeline.

Governance Intent: The machine-assisted translation of a Business Intent into governed protocol structure — the intermediate artifact between human goals and compiled canonical artifacts. Produced from the Business Intent using the Protocol Projection Surface as context; reviewed by humans before compilation.

Appendix B: Reference Implementation Notes

The conceptual model presented in this paper has been realized in the open-source Protocol-Governed Systems (PGS) reference implementation available on GitHub:

[PGS Workspace Repository](#)

The implementation serves as a practical realization of the concepts discussed throughout this paper, demonstrating how governance declarations can be compiled into executable protocol artifacts and enforced through a deterministic runtime. The examples, terminology, and architectural discussions in this paper are based on **PGS v0.4.0**, which represents the state of the project at the time of publication.

Since PGS is under active development, subsequent releases may introduce additional governance capabilities, authoring workflows, compiler validations, protocol artifacts, optimization strategies, and runtime features beyond those described here. Readers should therefore view this paper as a conceptual and architectural foundation rather than a complete description of all future implementation capabilities. For the latest documentation, releases, and implementation details, consult the project repository.

Appendix C: References

Ganti, B. (2026). *Protocol-Governed Systems: Conceptual Model*. DOI: <https://doi.org/10.5281/zenodo.20300611>

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

Lamport, L. (1994). The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems*, 16(3), 872–923.

Abadi, M., & Cardelli, L. (1996). *A Theory of Objects*. Springer-Verlag.